

Report

Question And Answer

6CS012

Artificial Intelligence and Machine Learning

Herald College Kathmandu

Student Information:

Name	Student Number	Group	Date
Pratik Karki	2408034	L6CG15	May 9, 2026

Table Of Content

Long Question..... 3

 Question 1:..... 3

 Answer:..... 3

Short Question.....4

 Question 1:..... 4

 Answer:..... 5

 Question 2:..... 5

Long Question

Question 1:

2. You are a Machine Learning Engineer at a rapidly growing e-commerce company responsible for deploying and maintaining ML systems in production.

- **Identify and explain at least three real-world challenges encountered in ML systems (e.g., data drift, model degradation, latency, scalability, monitoring).**
- **For each challenge:**
 - **Discuss potential consequences if not properly addressed.**
 - **Propose practical solutions (e.g., retraining pipelines, monitoring systems, MLOps practices, distributed serving).**
 - **Highlight any trade-offs involved in your solution.**
- **Reflect on how cross-functional collaboration (data scientists, engineers, product managers) contributes to building reliable ML systems.**
- **Briefly comment on how modern tools (e.g., automation, pipelines, or LLM-based systems) may help address these challenges.**

Answer:

In a rapid evolving environment of e-commerce, deployment of machine learning models goes beyond static code to managing dynamic systems. In this report, we identify three critical challenges which are Data drift, system latency and scalability in detail measuring their consequences, solutions and inherent trade-offs.

Data drift and concept drift are two of the main production issues with the ML system. Customer preferences, purchasing patterns, and market trends are constantly shifting in e-commerce platforms. A model's accuracy and prediction quality deteriorate if it is not updated frequently, which may lead to irrelevant product recommendations, decreased consumer engagement, decreased sales, and revenue loss. Businesses use automated retraining pipelines and continuous monitoring systems, such as Evidently AI, to identify statistical changes in data in order to overcome this problem. MLOps's methods aid in automating the deployment, retraining, and data validation procedures. However, the solutions come with large computational costs, extra

infrastructure needs, and the potential to retrain the model using noisy data or transient seasonal patterns, which could lower overall stability.

Inference latency, which occurs when the ML system takes too long to produce predictions during crucial user interactions like product search or checkout, is another issue. Even little delays can have a detrimental impact on the user experience on e-commerce platforms, which could result in increased bounce rates, abandoned shopping carts, and disgruntled customers. Model optimization methods like quantization and pruning, as well as distributed model serving systems like TorchServe or NVIDIA Triton inference server, are part of the solution to this problem. These technologies increase prediction speed and enable models to effectively manage many concurrent queries. Aggressive optimization has the disadvantage of potentially slightly lowering model accuracy, so businesses must strike a balance between forecast quality and speed.

Another issue is scalability and system reliability. E-commerce platforms often face sudden traffic spikes during special events, sales or promotions. There may be slowdowns, crashes or service outages if ML infrastructure cannot scale properly, affecting revenue and customer trust. To address this issue, companies use horizontal scaling. Distributed architectures help to maintain stable performance under heavy workloads. With benefits, it also has drawbacks, scalable systems increase infrastructure complexity, operational costs and maintenance overhead for engineering teams, requiring skilled professionals to manage deployment and monitoring.

In order to build reliable ML systems, cross-functional collaboration plays a vital role. Data scientists develop and evaluate models, software engineers focus on deployment and infrastructure, product managers ensure that ML systems align with business goals and customer needs. Communication between these teams helps to identify issues in the early stage, improve deployment reliability and ensures models deliver real business value.

Modern tools and automation technologies have improved the management of ML systems in production. Automated pipelines, CI/CD workflows, monitoring platforms, and MLOps frameworks help us to reduce manual effort, improve reproducibility, and speed up deployment cycles. LLM-based systems and AI-assisted monitoring tools are able to automate documentation, anomaly detection, and model evaluation. While these technologies increase efficiency and scalability, they also introduce challenges related to cost, governance, and dependency on complex tooling ecosystems.

Short Question

Question 1:

Overfitting is a common challenge in deep learning, especially when models have high capacity.

- Describe at least two techniques used to reduce overfitting (e.g., dropout, early stopping, data augmentation, L2 regularization).
- For each technique:
 - Explain the intuition behind how it works.
 - Describe how it affects the training process or model behavior.
 - Provide a real-world application example (e.g., image classification, text classification, fraud detection).

Answer:

A high-capacity model experiences overfitting when it memorizes noise from the training set, which results in poor generalization on new data. Dropout and data augmentation are two popular strategies to lessen this.

- Dropout: During each training iteration, the intuition entails "dropping out" neurons at random. By preventing units from co-adapting excessively, the network is forced to acquire more resilient features. In actuality, it functions as a type of ensemble learning by streamlining the model architecture during training. Real-world application: Dropout keeps the model from becoming overly dependent on particular "trigger phrases" that may only occur in the training set when it comes to text classification (such as hotel reviews).
- Data Augmentation: This method uses transformations like rotation, flipping, and zooming to boost the training set's diversity. The model intuitively learns that a "fruit" is a "fruit" regardless of its orientation. By offering more diverse samples without gathering fresh data, it greatly enhances model behavior. Use in the real world: By enriching a fruit dataset, CNN is able to identify an apple even when it is rotated or dimly lit.

Question 2:

Question 2:

Explain the fundamental differences between a Convolutional Neural Network (CNN) and a Recurrent

Neural Network (RNN):

- **In terms of:**
 - **Input structure and data type**
 - **Information flow and architecture design**
 - **Parameter sharing mechanism**
- **Provide suitable real-world examples where each architecture is preferred (e.g., image classification for CNNs, sequential/time-series modeling for RNNs).**
- **Briefly discuss common challenges encountered during training deep learning models, such as:**
 - **Vanishing or exploding gradients**
 - **Overfitting**
- **For each challenge, mention at least one mitigation technique (e.g., batch normalization, gradient clipping, early stopping).**

CNNs and RNNs differ mostly in how they handle data structurally. CNNs are made for spatial data, or images, and they scan 2D grids using filters. RNNs, on the other hand, use internal loops to retain recollection of prior inputs and are designed for sequential data (text/time-series).

- **Information Flow:** In order to extract local features, information passes through convolutional layers in CNNs' feed-forward flow. The output of a hidden state is fed back as input for the subsequent phase in an RNN's feedback cycle.
- **Parameter Sharing:** While RNNs share parameters across time (the same weight is applied at each stage of the sequence), CNNs share parameters across space (the same filter scans the whole image).
- **Difficulties:** Using LSTM units helps to avoid Vanishing Gradients, a typical problem in RNNs when long-range relationships are lost. Batch normalization helps reduce the overfitting that CNNs frequently experience.

